

Inheritance

Inheritance allows one object to **reuse properties and methods of another object**.

But here's the key truth:

JavaScript does NOT have classical inheritance (like Java/C++).

It uses **Prototype-Based Inheritance**.

The Core Idea

In JavaScript, objects don't copy from other objects.

They are **linked**.

This link is called the:

[[Prototype]] (a hidden internal reference)

Accessible as:

`obj.__proto__`

Real Meaning of Inheritance in JS

Instead of:

"Child gets a copy of Parent"

JavaScript does:

"Child asks Parent when it doesn't know something."

This is called:

Delegation (Not Copying)

Example Without Constructors (Pure Prototype Understanding)

```
const animal = {
```

```
  eats: true,
```

```
  walk() {
```

```
    console.log("Animal walks");  
  }  
};
```

```
const dog = Object.create(animal);
```

```
dog.bark = function () {  
  console.log("Dog barks");  
};
```

Now:

```
dog.walk(); // inherited  
dog.bark(); // own method
```

What Happens Internally?

dog → animal → Object.prototype → null

If JS doesn't find something in dog,
it looks inside animal.

That search process is called:

 **Prototype Chain Lookup**

How Constructor Inheritance Works

Let's now connect this to constructors (what interviews test).

Step 1 — Parent Constructor

```
function Animal(name) {  
  this.name = name;  
}
```

Step 2 — Parent Methods Go On Prototype

```
Animal.prototype.walk = function () {  
  console.log(this.name + " is walking");  
};
```

Why prototype?

Because methods must be shared, not duplicated.

Step 3 — Child Constructor

```
function Dog(name, breed) {  
  Animal.call(this, name); // inherit properties  
  this.breed = breed;  
}
```

Why .call()?

It runs the parent constructor like this:

Animal executed inside Dog

this → new Dog object

So Dog gets:

name property

This is called:

 Constructor Borrowing

Step 4 — Link Prototypes (Real Inheritance Happens Here)

```
Dog.prototype = Object.create(Animal.prototype);
```

Now:

Dog.prototype → Animal.prototype

This enables method inheritance.

Step 5 — Fix Constructor Reference

Dog.prototype.constructor = Dog;

Because step 4 overwrote it.

✓ Final Usage

const d1 = new Dog("Rocky", "Labrador");

d1.walk(); // inherited
console.log(d1.name);
console.log(d1.breed);

const d1 = new Dog();

JS builds this invisible chain:

d1
↓
Dog.prototype
↓
Animal.prototype
↓
Object.prototype
↓
null

When calling:

d1.walk();

JS searches step-by-step:

- 1** d1 → not found
- 2** Dog.prototype → not found
- 3** Animal.prototype → FOUND ✓

This search mechanism = Inheritance in JavaScript.

Types of Inheritance in JavaScript

1 Prototypal Inheritance (Native JS Way)

Using `Object.create()` directly.

2 Constructor Inheritance

Using `.call()` + prototype chaining.

3 ES6 Class Inheritance (Just Syntax Sugar)

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  
  walk() {  
    console.log("Walking");  
  }  
}
```

```
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name);  
    this.breed = breed;  
  }  
}
```

This is converted internally into the same prototype logic.

There is **no real class system** underneath.

Q.Why Not Put Methods Inside Constructor?

Because then inheritance breaks:

Each object would carry its own copy → memory waste.

Prototype enables **true sharing**.

Q.Why JavaScript Uses Prototype Instead of Classes?

Because JS was designed as a **dynamic language**:

- Objects can change at runtime
- Inheritance can be modified live
- Much more flexible than rigid class systems

Q. 1. What is Inheritance in JavaScript?

Expected Answer:

Inheritance in JavaScript is the ability of one object to access properties and methods of another object through the **prototype chain**.

Q.2 Why Do We Use Parent.call(this, ...) ?

To inherit **instance properties**.

Without it:

`this.name = name;`

would NOT exist inside Child.

call() runs the Parent constructor inside Child and binds:

this → Child instance

Q.3. Why Do We Use `Object.create(Parent.prototype)` Instead of:

`Child.prototype = Parent.prototype;` ❌

Because that would make both constructors share the SAME prototype object.

Problems:

- Changing Child affects Parent.
- Breaks encapsulation.

`Object.create()` creates a **new object linked to `Parent.prototype`**, forming a safe prototype chain.

Q.4 Why Do We Reset `Child.prototype.constructor` ?

After:

`Child.prototype = Object.create(Parent.prototype);`

the constructor incorrectly points to Parent.

So we fix it:

`Child.prototype.constructor = Child;`

This restores the correct identity:

`childObj.constructor === Child` ✅